

Deep Learning Final Project Report

Smart Waste Detection and Classification System

Submitted to: Prof. S.H. Shabbeer Basha

Course: Deep Learning (B.Tech CSE)

Academic Year: 2025–2026

Date: April 2026

Group name : Thanushree.NH - 165

Adarsh Kumar Singh-180

Syed Usman-172

Table of Contents

1. Introduction	4
2. Problem Statement	4
3. Dataset Description	5
4. Theoretical Background	6
4.1 Convolutional Neural Networks (CNNs)	6
4.2 Object Detection: One-Stage vs Two-Stage	7
4.3 YOLOv5 Architecture	7
4.4 YOLOv8 Architecture	8
4.5 Faster R-CNN Architecture	9
5. Data Preprocessing Pipeline	10
6. Model Training and Configuration	11
7. Results and Evaluation	13
8. Model Comparison	14
9. Real-World Inference	15
10. Conclusion and Future Work	15
11. References	16

Abstract

Improper waste segregation remains one of the leading causes of environmental degradation in urban India, with mixed waste streams significantly reducing recycling efficiency and increasing landfill burden. This project addresses that problem directly: we design, train, and evaluate a Smart Waste Detection and Classification System using deep learning-based object detection architectures.

Three architectures are implemented and compared on the same dataset: YOLOv5 (single-stage, CSPDarknet backbone), YOLOv8 (single-stage, C2f backbone with AdamW optimiser), and Faster R-CNN (two-stage, ResNet-50 + FPN backbone with transfer learning). The dataset consists of annotated waste images categorised into three classes: Wet Waste, Dry Waste, and Recyclable, originally labelled with polygon annotations that were converted to axis-aligned bounding boxes for compatibility across all three models. Class balancing was applied to the validation and test splits to ensure fair evaluation.

Both YOLO variants were trained for 100 epochs with cosine learning rate scheduling and mosaic augmentation. Faster R-CNN was trained for 20 epochs using SGD with step-wise learning rate decay. Real-world inference was demonstrated on unseen images from outside the training distribution. The project provides an architectural comparison of one-stage versus two-stage detectors in the context of a practical waste management application, with the goal of identifying the most deployable model for smart dustbin systems.

1. Introduction

Waste management is one of the most pressing urban challenges in India and globally. The inefficiency of manual waste segregation at the point of collection — where wet organic waste, dry recyclable waste, and general recyclable materials are routinely mixed — leads to contaminated waste streams that cannot be processed economically. Automated, vision-based waste detection systems mounted at collection points represent a scalable and cost-effective solution to this problem.

Deep learning, specifically Convolutional Neural Networks (CNNs) and their derivative architectures, has transformed the field of computer vision over the past decade. Object detection models — which simultaneously localise objects in an image with bounding boxes and classify those objects — are particularly well-suited to this problem. A camera mounted at a waste disposal point can capture an image of the waste, and a trained detection model can instantly identify whether the item is wet waste, dry waste, or a recyclable material, triggering the appropriate bin compartment to open.

This project implements and compares three state-of-the-art object detection architectures for this task: YOLOv5, YOLOv8, and Faster R-CNN. The choice of these three models is deliberate — YOLOv5 and YOLOv8 represent the evolution of single-stage detection (prioritising speed), while Faster R-CNN represents the mature two-stage detection paradigm (prioritising accuracy). Understanding the tradeoffs between these architectures is essential for selecting the right model for real-world deployment.

2. Problem Statement

2.1 The Core Challenge

The central challenge is to build a system that, given an image of waste, can: (a) detect the location of each waste item within the image using a bounding box, and (b) classify each detected item into one of three categories: Wet Waste, Dry Waste, or Recyclable.

The three classes are defined as follows:

- **Wet Waste** Organic, biodegradable waste such as food scraps, vegetable peels, cooked food, and garden waste. This class has the highest urgency for correct classification, as contamination of recyclables with wet waste renders them unprocessable.
- **Dry Waste** Non-recyclable dry waste such as used tissues, broken items, ceramic, thermocol, and soiled packaging that cannot be recycled.
- **Recyclable** Clean, recyclable materials including plastic bottles, glass, metal cans, paper, cardboard, and clean packaging.

2.2 Why This is a Hard Problem

Waste detection is particularly challenging compared to standard object detection datasets for several reasons. First, waste objects have highly irregular shapes and appearances — a plastic bag looks different when crumpled versus flat. Second, waste items are often partially occluded by other items in real dustbin scenarios. Third, the same category (e.g., recyclable plastic) can appear in many different forms, sizes, and lighting conditions. These factors make it essential to use robust augmentation strategies and well-balanced training data.

2.3 Expected Outcomes

At the conclusion of this project, the expected deliverables are:

- A trained YOLOv5 model capable of detecting and classifying waste into three categories with measurable precision and recall on a held-out test set.
- A trained YOLOv8 model trained under comparable conditions for direct architectural comparison.
- A trained Faster R-CNN model implemented in PyTorch from scratch, demonstrating understanding of the two-stage detection paradigm.
- A quantitative comparison of all three models on mAP@50, Precision, and Recall metrics.
- Real-world inference demonstrations on images outside the training distribution.

3. Dataset Description

3.1 Source and Format

The dataset used in this project is a custom waste detection dataset stored as a ZIP archive (waste_balanced_FIXED.zip). The dataset is organised in YOLO format with three splits: train, valid, and test. Each split contains an images/ folder and a corresponding labels/ folder.

Critically, the original annotation format was polygon-based — each label file contained polygon vertex coordinates describing the precise outline of each waste item. This format is incompatible with YOLOv5's bounding-box training pipeline, which requires annotations in the format: [class_id, x_center, y_center, width, height] (all normalised to [0,1]). A preprocessing step was therefore required to convert polygon annotations to axis-aligned bounding boxes.

3.2 Class Distribution

The dataset contains three classes, indexed as follows:

Class Index	Class Name	Training Samples (approx.)	Post-Balancing
0	Wet Waste	5,149	Balanced
1	Dry_Waste	5,121	Balanced
2	Recyclable	5,154	Balanced

The training split was already naturally balanced (approximately equal class distribution), which is excellent for training a classifier without class-weighted loss functions. The validation and test splits required rebalancing via a custom undersampling strategy.

3.3 Why Class Balancing Matters

An imbalanced dataset causes a model to develop a bias toward the majority class. For example, if 70% of validation images are Recyclable, a model that always predicts Recyclable would achieve 70% accuracy despite learning nothing meaningful. By ensuring equal representation of each class in the evaluation splits, we guarantee that the computed Precision and Recall values reflect genuine per-class discrimination ability.

4. Theoretical Background

4.1 Convolutional Neural Networks (CNNs)

A Convolutional Neural Network is a feedforward neural network specifically designed for processing grid-structured data such as images. Unlike fully-connected networks, which treat every pixel independently, CNNs exploit the spatial structure of images through three key operations:

- Convolution — a learnable filter slides across the input feature map, computing dot products to detect local patterns (edges, textures, shapes).
- Pooling — spatial downsampling that reduces the size of feature maps while retaining dominant activations, providing translational invariance.
- Non-linear Activation (ReLU) — introduces non-linearity by zeroing negative activations, allowing the network to learn complex, non-linear mappings.

Deep CNN architectures stack many such layers, with early layers detecting low-level features (edges, corners) and deeper layers combining them into semantic representations (object parts, whole objects). All three architectures used in this project use CNN backbones as their feature extractors.

4.2 Object Detection: One-Stage vs. Two-Stage Detectors

Object detection requires the model to both localise (where is the object?) and classify (what is the object?) every instance in an image. There are two dominant paradigms:

Two-Stage Detectors (e.g., Faster R-CNN):

In two-stage detection, the process is explicitly divided: Stage 1 generates a set of candidate regions (Region Proposals) that are likely to contain objects, using a sub-network called the Region Proposal Network (RPN). Stage 2 takes each proposal, extracts its features, and classifies it into a specific category while refining its bounding box. The advantage is high accuracy; the cost is speed, since every proposal must be processed independently.

One-Stage Detectors (e.g., YOLO family):

In one-stage detection, the entire image is processed once through the network, and object locations and classes are predicted simultaneously from a fixed grid of anchor-based predictions. The network divides the image into a grid (e.g., 80×80 , 40×40 , 20×20 at multiple scales) and at each grid cell predicts bounding boxes and class probabilities. This is significantly faster than two-stage methods, making it suitable for real-time deployment.

4.3 YOLOv5 Architecture

YOLOv5 (You Only Look Once, version 5) is implemented by Ultralytics and represents a highly optimised single-stage detection framework. In this project, the yolov5s (small) variant is used — the 's' denotes the smallest and fastest variant in the YOLOv5 family.

The YOLOv5 architecture consists of three main components:

- **Backbone (CSPDarknet)** Cross-Stage Partial Darknet extracts multi-scale feature representations from the input image. CSP connections reduce gradient redundancy during backpropagation, improving learning efficiency.
- **Neck (PANet)** Path Aggregation Network aggregates features from multiple backbone scales, allowing the model to detect objects of varying sizes using multi-scale feature maps.
- **Head** Three detection heads operating at different scales (large, medium, small) predict bounding box coordinates, objectness scores, and class probabilities for each anchor.

The total loss function in YOLOv5 is a weighted sum of three components:

$$L_{\text{total}} = \lambda_{\text{box}} \cdot L_{\text{box}} + \lambda_{\text{obj}} \cdot L_{\text{obj}} + \lambda_{\text{cls}} \cdot L_{\text{cls}}$$

- **L_{box} (Box Regression Loss)** — measures how accurately the predicted bounding box matches the ground truth box. Uses CIoU (Complete Intersection over Union) loss, which accounts for both overlap and aspect ratio.
- **L_{obj} (Objectness Loss)** — binary cross-entropy measuring whether the model correctly identifies grid cells that contain objects versus background.
- **L_{cls} (Classification Loss)** — binary cross-entropy per class measuring whether the detected object is assigned the correct category.

4.4 YOLOv8 Architecture

YOLOv8 is the latest generation of the Ultralytics YOLO family, introducing several architectural improvements over YOLOv5:

- **C2f Module (Cross-Stage Partial with two bottlenecks)** — replaces the C3 module from YOLOv5, providing richer gradient flow while maintaining computational efficiency.
- **Anchor-Free Detection Head** unlike YOLOv5 which uses predefined anchor boxes, YOLOv8 uses an anchor-free head that directly predicts object centre points and box dimensions. This reduces hyperparameter sensitivity.
- **Decoupled Head** separate heads for classification and box regression, which has been shown to improve convergence speed.

In this project, the AdamW optimiser is used for YOLOv8 training, in contrast to the SGD-based training in YOLOv5. AdamW adapts the learning rate for each parameter individually and includes decoupled weight decay, which generally provides faster convergence on complex tasks.

4.5 Faster R-CNN Architecture

Faster R-CNN (Region-based Convolutional Neural Network) is the foundational two-stage object detector used in this project. The implementation uses torchvision's `fasterrcnn_resnet50_fpn`, which consists of:

- **Backbone (ResNet-50 + FPN)** — ResNet-50 extracts deep features using residual connections. The Feature Pyramid Network (FPN) on top creates a multi-scale feature hierarchy, allowing detection of objects at different resolutions.
- **Region Proposal Network (RPN)** — A small fully-convolutional network that slides over the feature map and proposes candidate object regions (called anchors). It predicts two values per anchor: an objectness score (is this anchor likely to contain an object?) and a bounding box refinement delta. This is the 'Stage 1' of the two-stage pipeline.
- **RoI (Region of Interest) Pooling** — Takes each proposal from the RPN and extracts a fixed-size feature map from the backbone feature map, regardless of the proposal's size.
- **Detection Head (FastRCNNPredictor)** — A fully-connected network that takes each RoI's features and predicts the final class label and refined bounding box. This is 'Stage 2'.

The model is initialised with ImageNet pretrained weights (transfer learning), and only the final detection head is replaced with a new predictor for our 4-class problem (3 waste classes + 1 background class).

The training loss for Faster R-CNN comprises four components tracked in this project:

- `loss_classifier` — Cross-entropy loss on the final object class predictions from Stage 2.
- `loss_box_reg` — Smooth-L1 regression loss on the final refined bounding box coordinates from Stage 2.
- `loss_objectness` — Binary cross-entropy loss from the RPN (Stage 1) measuring whether each anchor is correctly identified as containing an object or not.
- `loss_rpn_box_reg` — Smooth-L1 loss from the RPN on the coarse bounding box refinement in Stage 1.

5. Data Preprocessing Pipeline

The preprocessing pipeline is a critical and non-trivial component of this project. Three distinct preprocessing steps were required before any model could be trained.

5.1 Polygon-to-Bounding-Box Conversion

The original dataset annotations were stored in polygon format — each annotation was a sequence of (x, y) vertex coordinates tracing the precise outline of each object. YOLO-family models, however, require axis-aligned rectangular bounding boxes in normalised [x_center, y_center, width, height] format.

The conversion algorithm implemented in the YOLOv5 notebook computes the minimum bounding box that encloses each polygon:

```
x_center = (min(x_coords) + max(x_coords)) / 2
y_center = (min(y_coords) + max(y_coords)) / 2
width = max(x_coords) - min(x_coords)
height = max(y_coords) - min(y_coords)
```

Coordinate clamping is applied to ensure all values remain in the valid [0, 1] range, and a minimum bounding box size of 0.001 is enforced to prevent degenerate zero-area boxes that would cause training instability.

Importantly, this conversion step is only required for YOLOv5. YOLOv8 (via the Ultralytics API) and Faster R-CNN (which uses pixel-coordinate bounding boxes) each have their own annotation handling. For Faster R-CNN, the WasteDataset class in the notebook performs an inverse conversion from YOLO normalised format to pixel coordinates: $x1 = (cx - w/2) \times \text{image_width}$.

5.2 Class Rebalancing

Although the training split was naturally balanced, the validation and test splits showed class imbalance. A custom rebalance() function was implemented that:

1. Scans all label files in the split and assigns each image to its dominant class (the class with the most annotation instances).
2. Identifies the minimum count across all classes (min_n).
3. Randomly samples exactly min_n images from each class using a fixed random seed (42) for reproducibility.
4. Copies the selected images and labels to a new output directory.

This ensures that the evaluation metrics are not artificially inflated by majority-class bias.

5.3 Data Augmentation

Data augmentation is applied during training to improve generalisation. The YOLOv5 training uses a custom hyperparameter file specifying the following augmentations, each relevant to the waste detection task:

- Mosaic (1.0) — Combines four training images into one, dramatically increasing the variety of object scales and positions per training step. This is particularly beneficial for waste detection where objects vary widely in size.
- MixUp (0.1) — Blends two images together with a mixing coefficient, creating soft training targets that improve calibration.
- Horizontal/Vertical Flip (0.5 / 0.3) — Waste items have no canonical orientation, so flipping is a natural and valid augmentation.
- HSV Augmentation — Random adjustments to hue (0.015), saturation (0.7), and value (0.4) simulate different lighting conditions, improving robustness to illumination variation.
- Scale (0.5) and Shear (2.0) — Simulate perspective and distance variations.

6. Model Training and Configuration

6.1 YOLOv5 Training Configuration

YOLOv5 was trained using the official Ultralytics YOLOv5 training script with the following configuration:

Hyperparameter	Value
Model Variant	yolov5s (small)
Input Image Size	640 × 640 px
Batch Size	32
Epochs	100 (early stopping patience: 20)
Initial Learning Rate (lr0)	0.001
Final LR (lrf)	0.01 (cosine LR schedule)
Momentum	0.937
Weight Decay	0.0005
Warmup Epochs	5
Pretrained Weights	COCO (yolov5s.pt)

The cosine learning rate schedule (--cos-lr flag) causes the learning rate to decrease following a cosine curve from lr0 to $lr0 \times lrf$, providing fast initial learning followed by fine-grained convergence. The early stopping patience of 20 epochs prevents overfitting by halting training if mAP@50 on the validation set does not improve for 20 consecutive epochs.

6.2 YOLOv8 Training Configuration

YOLOv8 was trained using the Ultralytics Python API with the following configuration:



Hyperparameter	Value
Model Variant	yolov8s (small)
Input Image Size	640 × 640 px
Batch Size	32
Epochs	100 (early stopping patience: 20)
Optimiser	AdamW
Initial Learning Rate (lr0)	0.001
Cosine LR Schedule	Enabled
Warmup Epochs	5
Augmentation	Enabled (augment=True)
Pretrained Weights	COCO (yolov8s.pt)



The primary distinction from YOLOv5 is the use of the AdamW optimiser. Unlike vanilla SGD (used in YOLOv5 training), AdamW maintains individual adaptive learning rates for each parameter and applies decoupled weight decay. In practice, AdamW often converges faster, particularly in the early training epochs.

6.3 Faster R-CNN Training Configuration

Hyperparameter	Value
Backbone	ResNet-50 + FPN (pretrained=True)
Number of Classes	4 (3 waste + 1 background)
Batch Size	4
Epochs	20
Optimiser	SGD (momentum=0.9)
Initial Learning Rate	0.005
LR Schedule	StepLR (gamma=0.5, step=7)
Weight Decay	0.0005
Transfer Learning	Yes — ImageNet pretrained backbone

An important implementation note: Faster R-CNN treats background as a class, so the number of output classes is 4 (three waste categories + background), whereas YOLO uses implicit background modelling through objectness scoring. The batch size is significantly smaller (4 vs. 32) due to the higher per-image memory requirements of the two-stage pipeline.

Transfer learning is applied by initialising the backbone with ImageNet pretrained weights (pretrained=True). This provides the network with strong low-level and mid-level feature detectors learned from millions of images, dramatically reducing the number of epochs needed to converge on the relatively smaller waste detection dataset.

7. Results and Evaluation

7.1 Evaluation Metrics

All three models are evaluated on the same held-out test set using the following standard object detection metrics:

- Precision — Of all the detections the model made, what fraction were correct? High precision means few false alarms.
- Recall — Of all the ground-truth objects that exist, what fraction did the model successfully detect? High recall means few missed detections.
- mAP@50 (mean Average Precision at IoU threshold 0.5) — The primary benchmark metric. A detection is counted as correct only if its predicted bounding box overlaps the ground truth box by at least 50% ($\text{IoU} \geq 0.5$). mAP is the mean of Average Precision scores across all classes.

For the Faster R-CNN model, a custom evaluation function was implemented that matches predictions to ground truth using IoU-based matching (threshold = 0.5), correctly computing per-class TP, FP, and FN counts.

7.2 Training Curves

Both YOLO models produce training result CSV files (results.csv) that log per-epoch loss and metric values. The following quantities were tracked and plotted:

- Training Box Loss — should decrease monotonically as the model learns to localise objects.
- Training Objectness Loss — should decrease as the model learns to distinguish object regions from background.
- Training Classification Loss — should decrease as the model learns to assign correct class labels.

- Validation Precision and Recall — should both increase toward their respective plateaus.
- Validation mAP@50 — the primary indicator of model quality; should increase and plateau without significant overfitting.

For Faster R-CNN, the training loop separately records `total_loss`, `box_reg_loss`, `classifier_loss`, and `rpn_objectness_loss` at each epoch. The separation of these four loss components provides insight into which part of the two-stage pipeline is converging — for instance, if `rpn_objectness_loss` is low but `classifier_loss` remains high, it indicates the region proposals are good but the final classification is struggling.

7.3 Confusion Matrix Analysis

A confusion matrix was generated for each model on the validation set. The confusion matrix is a $C \times C$ matrix (where $C = 3$) where entry $[i][j]$ represents the number of ground-truth class- i objects that were predicted as class j . The diagonal represents correct predictions; off-diagonal entries represent misclassifications.

For the Faster R-CNN model, the confusion matrix is built from IoU-matched predictions, ensuring that only detections with sufficient spatial overlap with a ground-truth box are credited as correct classifications. This makes the matrix a true measure of classification quality, not merely detection recall.

8. Model Comparison

8.1 Architectural Comparison

Property	YOLOv5s	YOLOv8s	Faster R-CNN
Detection Type	One-Stage	One-Stage	Two-Stage
Backbone	CSPDarknet	C2f	ResNet-50 + FPN
Anchor Strategy	Anchor-Based	Anchor-Free	Anchor-Based (RPN)
Optimiser Used	SGD	AdamW	SGD
Batch Size	32	32	4
Epochs Trained	100	100	20
Transfer Learning	COCO	COCO	ImageNet
Speed (relative)	Fast	Fastest	Slow
Accuracy (relative)	Good	Better	Highest

8.2 Discussion: Which Model to Deploy?

For a real-world smart dustbin deployment, the choice of model depends on the deployment constraints. Three scenarios are considered:

- Edge Device (Raspberry Pi / Jetson Nano) — YOLOv5s or YOLOv8s are strongly preferred. Their single-pass inference completes in milliseconds on a GPU and achieves real-time performance even on CPU. Faster R-CNN's two-stage pipeline would introduce unacceptable latency on resource-constrained edge hardware.
- Server-Side API (Cloud inference) — Faster R-CNN is viable and would likely provide the highest accuracy, particularly for distinguishing visually similar classes (e.g., dry waste vs. recyclable packaging). The latency of a network call to a cloud API absorbs the slower inference time.
- Balanced Deployment (Jetson AGX / equivalent) — YOLOv8s with the anchor-free head represents the best tradeoff: modern architecture, fast inference, and improved accuracy over YOLOv5s without the overhead of two-stage processing.

Based on the project objective — deployment in smart dustbins for real-time waste sorting — YOLOv8s is the recommended production model. It provides the best balance of speed, accuracy, and ease of deployment via the Ultralytics API.

9. Real-World Inference

All three models were tested on images sourced from outside the training distribution — real-world photographs of overflowing dustbins, mixed waste, and street-side waste scenarios obtained from stock photography sources. This out-of-distribution testing is a critical validation step: a model that achieves high test set accuracy but fails on real-world images has overfit to the specific characteristics of its training dataset.

Key observations from real-world inference:

- YOLOv5 and YOLOv8 — Both models demonstrated detection capability on real-world images when run with a confidence threshold of 0.25. The anchor-based prior from COCO pretraining provided a useful initialisation for detecting objects of typical real-world sizes.
- Faster R-CNN — Tested with a confidence threshold of 0.5. The IoU-matched evaluation logic and the per-image prediction visualisation with colour-coded bounding boxes (blue for Wet Waste, green for Dry Waste, orange for Recyclable) provided clear, interpretable outputs.
- Common failure modes — In cluttered scenes with heavily occluded waste items, all three models occasionally produced duplicate or missed detections. This is a known limitation of single-image inference without temporal context or depth information.

10. Conclusion and Future Work

10.1 Conclusion

This project successfully designed, implemented, and evaluated a Smart Waste Detection and Classification System using three deep learning architectures: YOLOv5, YOLOv8, and Faster

R-CNN. All three models were trained on a balanced three-class waste dataset with a comprehensive preprocessing pipeline that included polygon-to-bounding-box annotation conversion, class rebalancing, and data augmentation.

The project demonstrates a clear understanding of the fundamental difference between one-stage and two-stage object detection paradigms, and provides practical evidence of the speed-accuracy tradeoff between these approaches. The implementation of a full custom training and evaluation pipeline for Faster R-CNN — including a custom PyTorch Dataset class, IoU-based evaluation function, and confusion matrix computation — reflects a deep engagement with the underlying mechanics of the two-stage detection framework, beyond simply using a high-level API.

10.2 Future Work

Several directions exist for extending this project:

- Instance Segmentation — Replace bounding-box detection with pixel-level segmentation using YOLOv8-seg or Mask R-CNN, enabling more precise waste boundary identification useful for robotic sorting arms.
- Dataset Expansion — Incorporate additional waste categories (e-waste, hazardous waste, medical waste) and increase dataset size to improve generalisation.
- Real-Time Video Deployment — Deploy the YOLOv8s model on an NVIDIA Jetson device with live camera input, implementing a servo-controlled bin actuation system triggered by detection outputs.
- Model Compression — Apply knowledge distillation or quantisation (INT8) to reduce model size for ultra-low-power microcontroller deployment.
- Active Learning — Collect hard examples from real-world deployment where model confidence is low, and use them to iteratively improve the model.

11. References

1. Classification of Recyclable Waste Using YOLO Models. (Paper 1 — details from Canva literature review slide). YOLO variants (YOLOv3, YOLOv4, YOLOv5) for recyclable waste detection.
2. Intelligent Waste Management System Using Deep Learning. (Paper 2 — details from Canva literature review slide). CNN-based waste classification system for automated sorting.
3. Intelligent Waste Classification System Using CNN. (Paper 3 — details from Canva literature review slide). ResNet-50 + SVM pipeline for waste identification.